

DeepSeek使用与提示词工程



>> 今天的学习目标

DeepSeek使用

- DeepSeek的创新
- CASE：小球碰撞试验（Cursor + DeepSeek-R1）
- DeepSeek私有化部署选择
- Ollama部署DeepSeek-R1
- 不同尺寸大模型选择
- API调用DeepSeek

提示词工程

- Prompt工程：设计与优化

Prompt原理

提示词关键原则

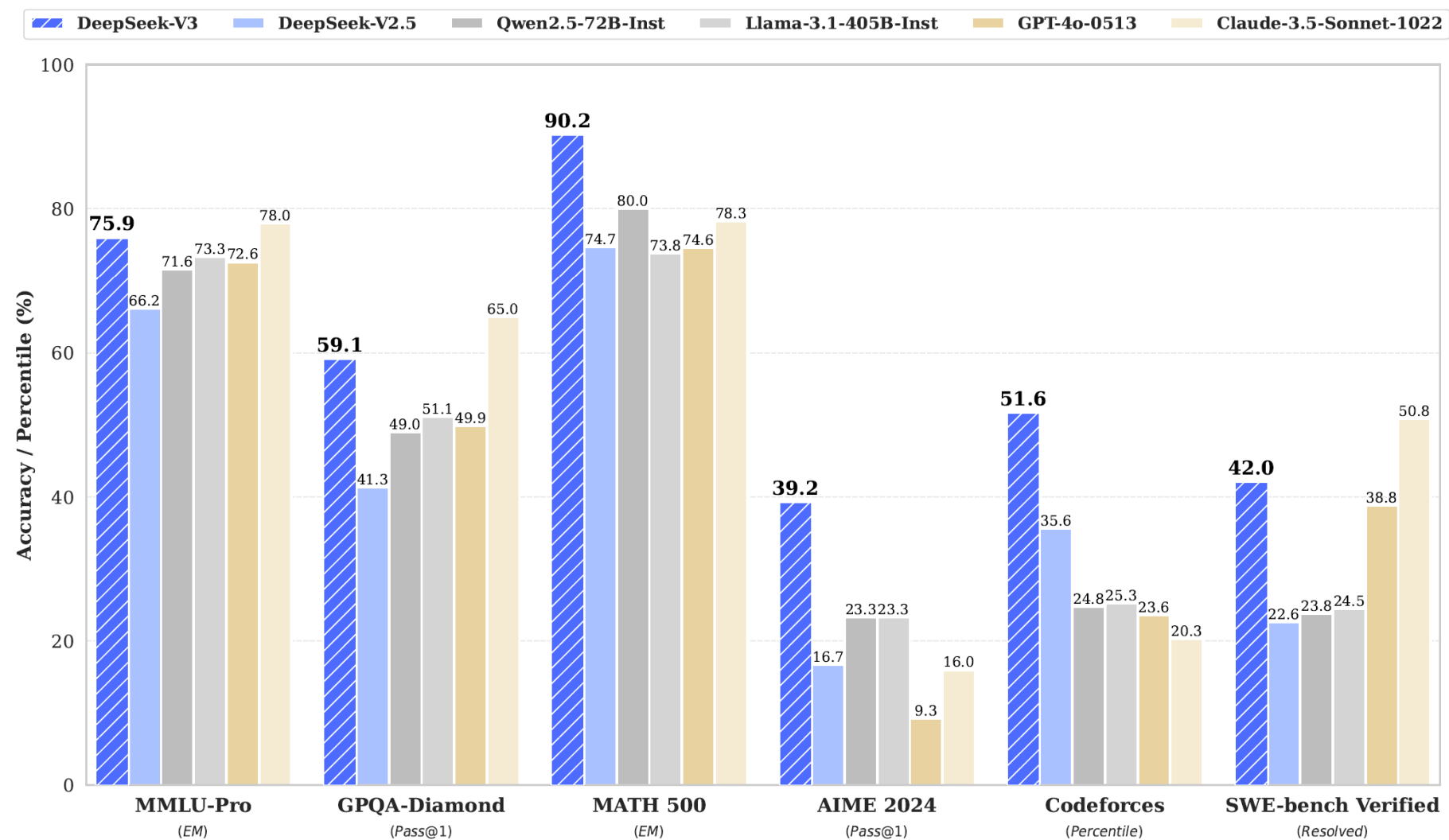
提示词编写框架（重要性排序）

提示词编写技巧（限制格式、区分、小样本学习、CoT、角色扮演等）

- Prompt工程实战

DeepSeek的创新

DeepSeek-V3模型



DeepSeek-V3 在推理速度上相较历史模型有了大幅提升。在目前大模型主流榜单中，DeepSeek-V3 在开源模型中位列榜首，与世界上最先进的闭源模型不分伯仲。

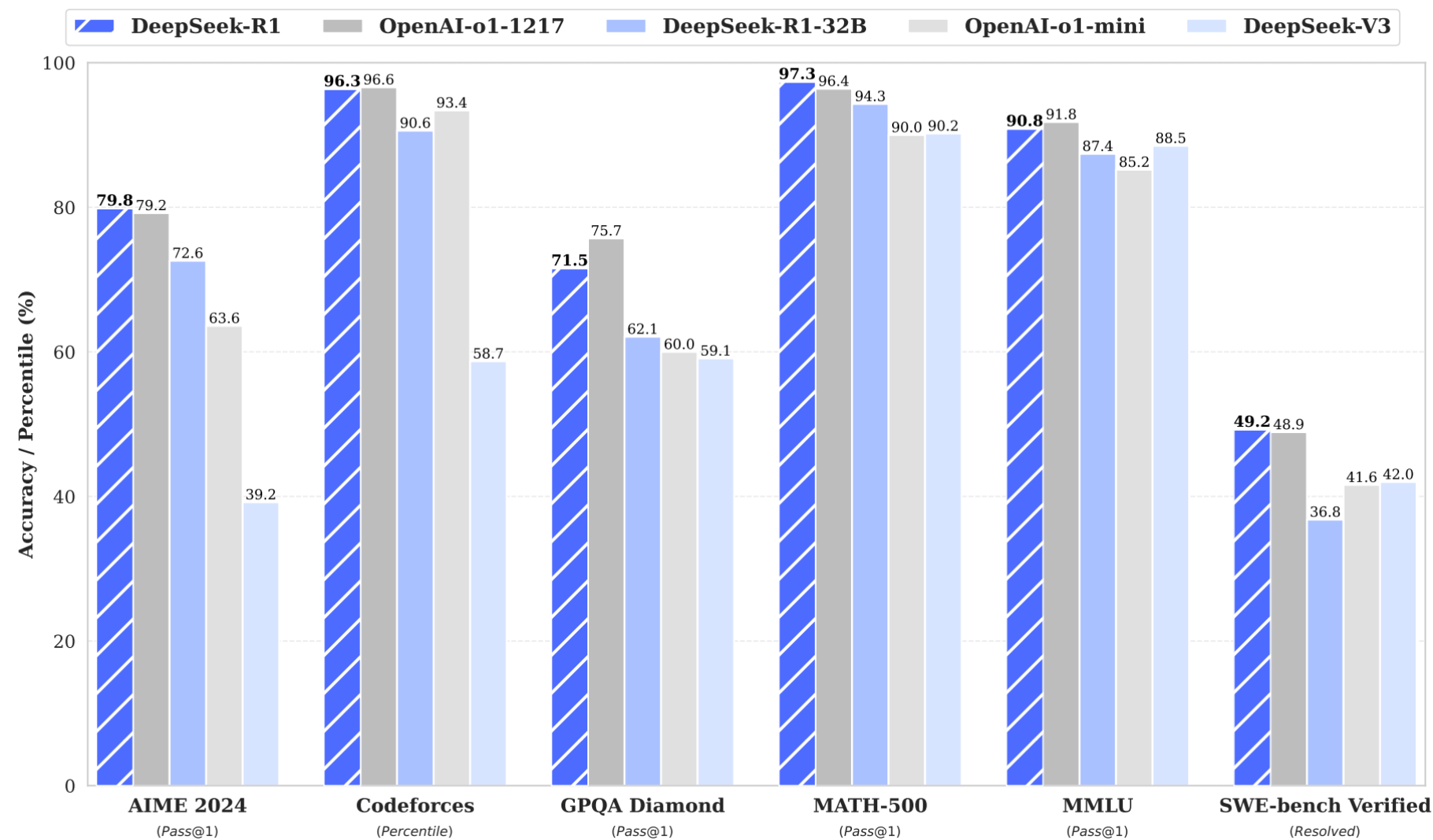
DeepSeek-V3的训练成本

Training Costs	Pre-Training	Context Extension	Post-Training	Total
in H800 GPU Hours	2664K	119K	5K	2788K
in USD	\$5.328M	\$0.238M	\$0.01M	\$5.576M

DeepSeek-V3 的训练成本，假设H800的租用是 \$2/小时

DeepSeek-V3的推出是2024年12月，并没有太大波澜
DeepSeek-R1火出圈，通过新的奖励机制GRPO (group relative policy optimization)，并使用规则类验证机制自动对输出进行打分。以V3为基础模型，一个多月内训练出了性能堪比GPT-o1的R1模型，成果非常亮眼。

DeepSeek-R1模型



DeepSeek-R1 完全对标 OpenAI-o1，与之前的DeepSeek-V3有大幅提升

DeepSeek-R1模型

DeepSeek-R1

DeepSeek-R1 遵循 MIT License，允许用户通过蒸馏技术借助 R1 训练其他模型。

DeepSeek-R1 上线 API，对用户开放思维链输出，通过设置 `model='deepseek-reasoner'` 即可调用。

2024-12-26 发布V3

2025-1-15 发布APP

2025-1-20 发布R1

多家企业宣布融合DeepSeek

.....

MIT License是一种非常宽松的开源许可协议，允许用户自由地使用、修改、分发和商业化软件或模型。相比之下，Meta Llama的License相对严格，虽然LLaMA3是开源的，但许可协议限制了商业用途和对模型的修改，比如新的模型如果使用 LLaMA，需要名称上带有LLaMA标识。

蒸馏小模型超越 OpenAI o1-mini

在开源 DeepSeek-R1-Zero 和 DeepSeek-R1 两个 660B 模型的同时，通过 DeepSeek-R1 的输出，蒸馏了 6 个小模型，其中 32B 和 70B 模型在多项能力上实现了对标 OpenAI o1-mini 的效果。

	AIME 2024 pass@1	AIME 2024 cons@64	MATH- 500 pass@1	GPQA Diamond pass@1	LiveCodeBench pass@1	CodeForces rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759.0
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717.0
o1-mini	63.6	80.0	90.0	60.0	53.8	1820.0
QwQ-32B	44.0	60.0	90.6	54.5	41.9	1316.0
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954.0
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189.0
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481.0
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691.0
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205.0
DeepSeek-R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633.0

DeepSeek的创新： MLA

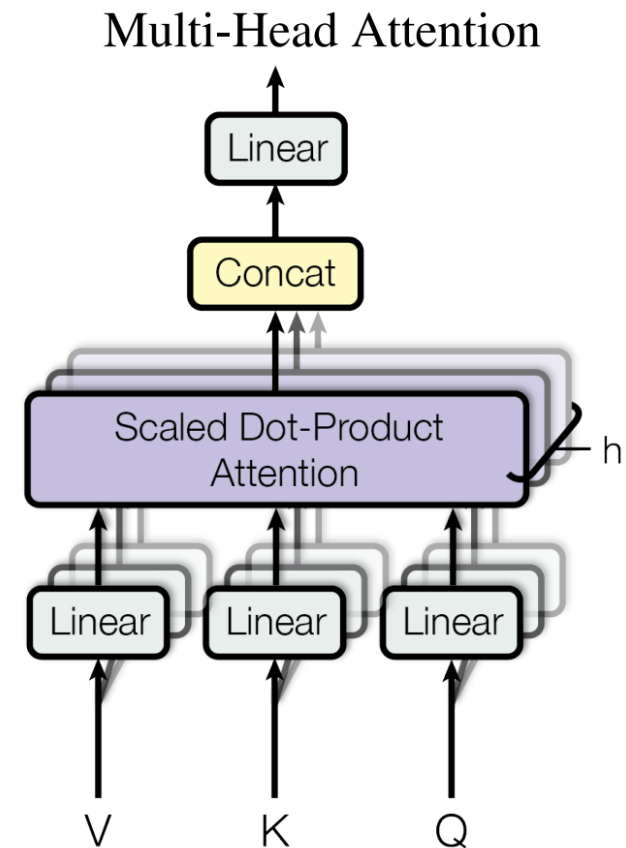
- MLA (Multi-Head Latent Attention)

在“All you need is attention”的背景下，传统的多头注意力（MHA, Multi-Head Attention）的键值（KV）缓存机制事实上对计算效率形成了较大阻碍。缩小KV缓存（KV Cache）大小，并提高性能，在之前的模型架构中并未很好的解决。

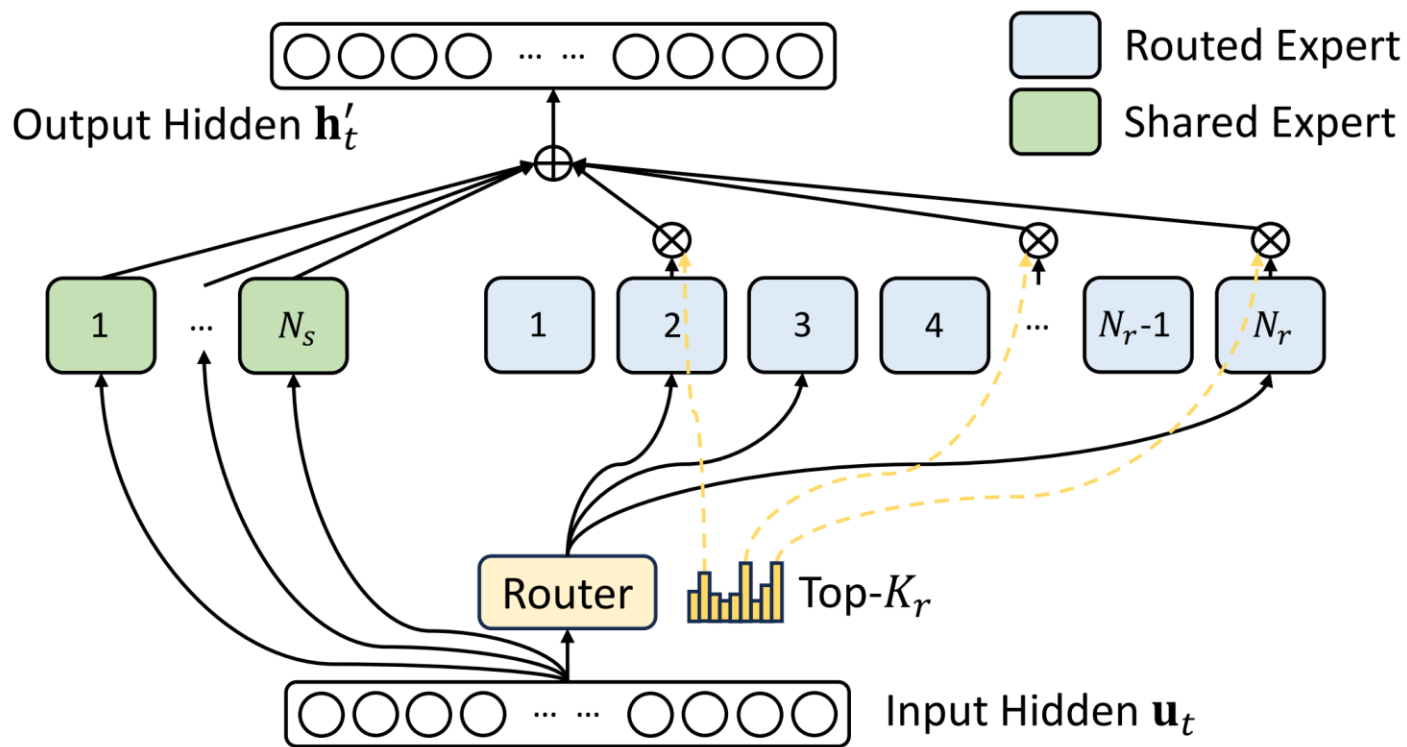
DeepSeek引入了MLA，一种通过低秩键值联合压缩的注意力机制，在显著减小KV缓存的同时提高计算效率。

低秩近似是快速矩阵计算的常用方法，在MLA之前很少用于大模型计算。

从大模型架构的演进情况来看，Prefill和KV Cache容量瓶颈的问题正一步步被新的模型架构攻克，巨大的KV Cache正逐渐成为历史。（实际上在2024年6月发布的DeepSeek-V2就已经很好的降低了KV Cache的大小）



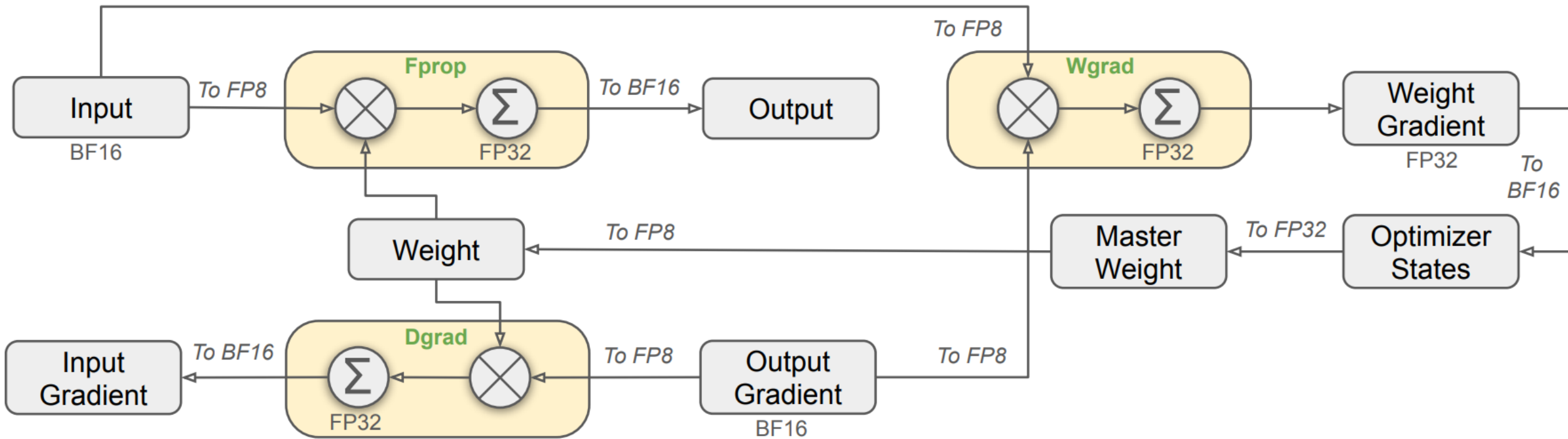
DeepSeek的创新: DeepSeek-MoE



V3使用了61个MoE (Mix of Expert 混合专家) block, 虽然总参数量很大, 但每次训练或推理时只激活了很少链路, 训练成本大大降低, 推理速度显著提高。

MoE类比为医院的分诊台, 在过去所有病人都要找全科医生, 效率很低。但是MoE模型相当于有一个分诊台, 将病人分配到不同的专科医生那里。DeepSeek在这方面也有创新, 之前分诊是完全没有医学知识的保安, 而现在用的是有医学知识的本科生来处理分流任务

DeepSeek的创新：混合精度框架



整个混合精度框架使用了FP8数据格式，但为了简化说明，只展示了线性算子（Linear Operator）的部分

采用了混合精度框架，即在不同的区块里使用不同的精度来存储数据。我们知道精度越高，内存占用越多，运算复杂度越大。DeepSeek在一些不需要很高精度的模块，使用很低的精度FP8储存数据，极大的降低了训练计算量。

Summary

Thinking: 为什么DeepSeek计算速度快，成本低？

- 架构设计方面

DeepSeek MoE架构：在推理时仅激活部分专家，避免了激活所有参数带来的计算资源浪费。

MLA架构：MLA通过降秩KV矩阵，减少了显存消耗。

- 训练策略方面

多token预测（MTP）目标：在训练过程中采用多token预测目标，即在每个位置上预测多个未来token，增加了训练信号的密度，提高了数据效率。

混合精度训练框架：在训练中，对于占据大量计算量的通用矩阵乘法（GEMM）操作，采用FP8精度执行。同时，通过细粒度量化策略和高精度累积过程，解决了低精度训练中出现的量化误差问题。

Summary

Thinking: 为什么DeepSeek-R1的推理能力强大?

- **强化学习驱动**: DeepSeek-R1通过大规模强化学习技术显著提升了推理能力。在数学、代码和自然语言推理等任务上表现出色, 性能与OpenAI的o1正式版相当。
- **长链推理 (CoT) 技术**: DeepSeek-R1采用长链推理技术, 其思维链长度可达数万字, 能够逐步分解复杂问题, 通过多步骤的逻辑推理来解决问题

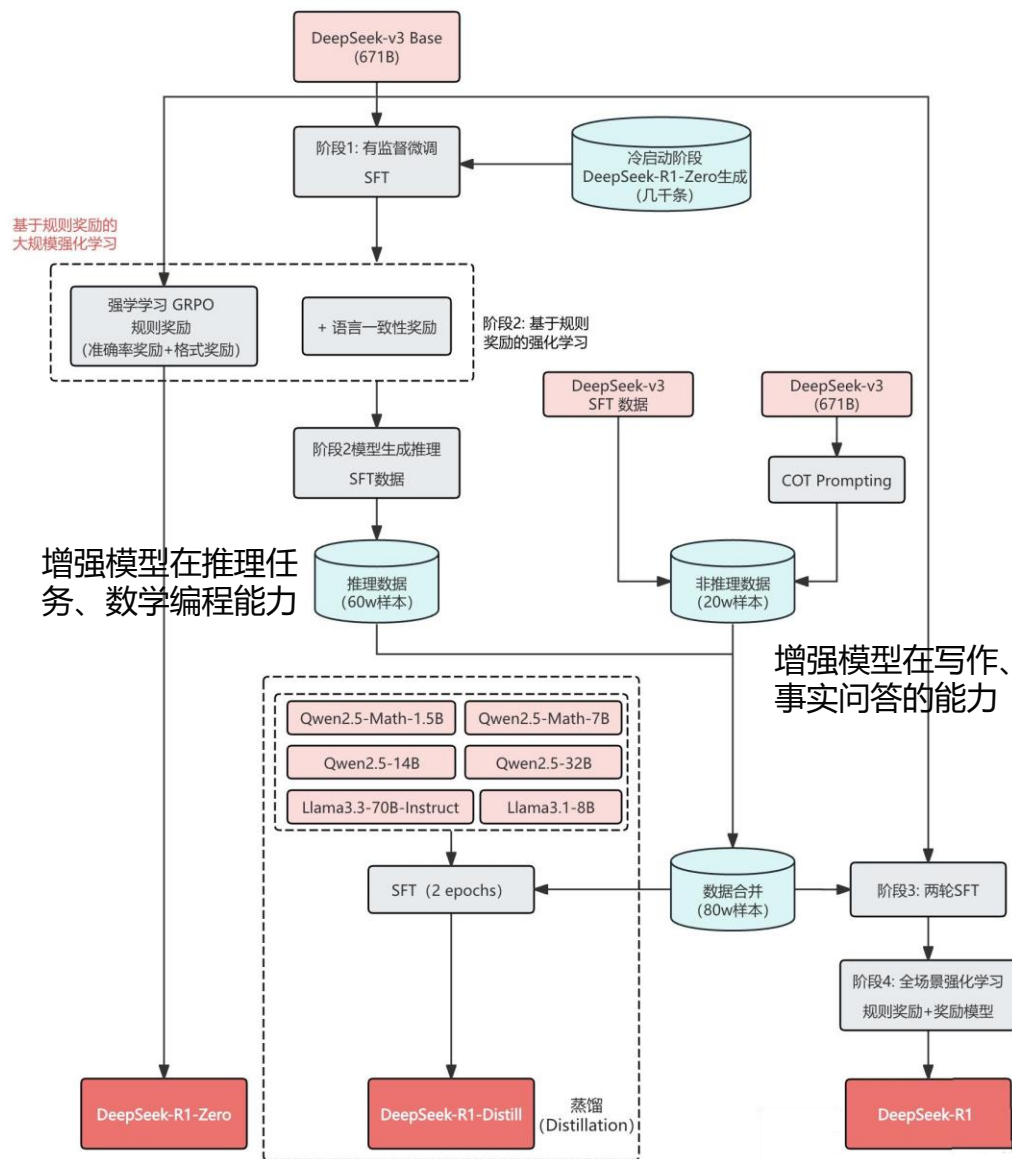
强化学习的作用: 训练大模型, 结合少量SFT, 引入少量高质量监督数据 (如数千个CoT示例) 进行微调, 提升模型初始推理能力, 再通过RL进一步优化, 最终达到与OpenAI o1相当的性能

长链推理CoT: CoT让AI模型逐步分解复杂问题, 比如在智能客服、市场分析报告、AI辅助编程领域

Summary

模型	方法
DeepSeek-R1-Zero	纯强化学习
DeepSeek-R1	冷启动 SFT -> RL -> COT + 通用数据 SFT (80w) ->全场景 RL
蒸馏小模型	直接用上面的 80w 数据进行SFT

DeepSeek-R1-Zero首次验证了纯强化学习在 LLM 中能显著增强推理能力的可行性，即无需SFT，仅通过 RL 即可激励模型学会长链推理和反思。提出了多阶段训练策略（冷启动->RL->SFT->全场景 RL），有效兼顾准确率与可读性，产出 DeepSeek-R1，性能比肩 OpenAI-o1-1217。展示了知识蒸馏在提升小模型推理能力方面的潜力，并开源多个大小不一的蒸馏模型（1.5B~70B）



不同尺寸的LLM

模型尺寸	参数量	特点	适用场景	硬件配置
1.5B	15亿	轻量级，快速响应	轻量级任务（如短文本生成、基础问答）	4核处理器、8GB内存，无需显卡
7B/8B	70-80亿	平衡型，性能适中	中等复杂度任务（如文案撰写、代码生成）	8核处理器、16GB内存， RTX 3060/4060
14B	140亿	高性能，适合复杂任务	数学推理、代码生成、长文本处理	i9-13900K、32GB内存， RTX 4090/A5000
32B	320亿	专业级，高精度任务	医疗/法律咨询、大规模训练、金融预测	Xeon 8核、128GB内存， 2-4张A100
70B	700亿	顶级模型，大规模计算	高精度专业领域任务、多模态任务预处理	8张A100/H100， 128GB内存
671B	6710亿	满血版，接近人类专家级推理能力	前沿科学研究、复杂商业决策分析	8张A100/H100（80GB）， 极高硬件需求

不同尺寸DeepSeek适合的任务

不同尺寸的LLM (DeepSeek)

- 小模型 (1.5B-14B)

响应快、硬件需求低，但基础能力薄弱（如7B模型在基础文本生成任务中表现不稳定，甚至“不及格”），无法胜任复杂任务。

- 中尺寸 (32B及以上)

性能接近满血版（32B约实现671B的90%性能），可满足专业领域需求，但本地化部署成本较高（需64GB内存、80GB显存等）。

- 满血版 (671B)

性能最强，适合超大规模AI研究、AGI 探索，但部署成本极高（需多节点分布式训练，硬件需求极高）

不同尺寸的LLM (Qwen3)

不同尺寸的 Qwen3 模型适用于不同场景：

- 0.6B/1.7B 适合本地测试、科研或边缘设备（如工控机）部署，
- 4B 适用于手机端应用，
- 8B 适用于电脑或汽车端的对话系统、语音助手等，
- 14B/32B 适合企业复杂任务落地，
- 30B-A3B 和 235B-A22B 则分别适合云端高效部署和旗舰级高性能场景（如数学证明、代码生成等）

235B-A22B在基准测试中可与 DeepSeek-R1、Grok-3 等顶尖模型竞争。

Qwen3 模型支持“思考模式”（逐步推理）和“非思考模式”（快速响应），并支持 119 种语言和方言，具备更强的 Agent 能力和原生 MCP 支持。

蒸馏的定义与方法

Thinking: 什么是蒸馏?

把大模型（教师）学到的知识压缩进小模型（学生），让后者在体量更小、推理更快的同时保留主要能力。

常用的方法

- 软标签蒸馏：用教师输出的概率分布训练学生；
- 隐藏层蒸馏：对齐中间表示；
- 数据增强：教师生成伪数据再训练。

蒸馏的定义与方法

Thinking：模型蒸馏的效果如何，在哪些场景中使用？

7B参数学生经13B教师蒸馏，C-Eval提升4.3分，推理延迟降至1/3，显存占用减半。

使用场景：端侧部署、高并发客服、低算力边缘设备，需快速响应且可牺牲少量精度。

模型蒸馏中需要注意：

- 教师-学生任务分布需一致；
- 温度系数别过大；
- 蒸馏后仍需微调避免灾难遗忘；
- 留5%数据做校验防退化。

量化的定义与方法

量化 (Quantization) :

指通过降低模型参数的存储精度（如从32位浮点数FP32降至8位整数INT8或4位INT4），以减少模型体积、加速推理并降低计算资源需求。

核心原理：

- 精度压缩：减少每个参数的比特数，例如FP32→INT8可减少75%存储需求。
- 误差补偿：如GPTQ采用逐层量化并调整剩余权重，以减少信息损失。
- 硬件加速：低精度计算（如INT8）在GPU/TPU上通常比FP32更快

量化的定义与方法

任务类型	FP16 vs. 4-bit 差异示例	错误表现
① 数值计算	问：“123456×789012 等于多少？”	FP16 全对； 4-bit 把中间步骤的进位算错，结果差 3 ~ 4 位
② 闭卷知识问答	问：“2022 年诺贝尔化学奖三位得主分别是谁？”	FP16 精准列出 Carolyn Bertozzi 等三人； 4-bit 漏掉一人或名字拼写错误
③ 长文本定位	给一篇 4k token 法律合同，问第 2 页第 3 条违约责任金额	FP16 能定位“在第 2 页第 3 条，违约金为合同总价的 5%”； 4-bit 给出 3% 这类相近却错误的数字
④ 少样本分类	5-shot 新闻分类（体育/财经/科技）	FP16 准确率 94%； 4-bit 掉到 78%，把“财经融资”错标成“科技”
⑤ 创意写作	写一首五言绝句，要求押“春”韵	FP16 押韵且意境连贯； 4-bit 出现“春/唇/醇”混押或平仄失调

量化越低，模型越小、越快，但越可能在需要精细数值推理或知识细节的任务上出错。

量化的定义与方法

- GPTQ：高效4-bit量化方法，逐层优化权重以减少误差，显著压缩模型（如175B→20GB），但推理时损失生成质量。
- Unsloth动态量化：混合精度方案，关键层保留FP16，其余4-bit。平衡性能与压缩率，适合需要高精度的任务（如数学推理）。
- QLoRA（微调量化）：训练时4-bit量化+LoRA适配器，微调后恢复FP16推理。降低训练成本，但部署时仍需FP16，适合微调大模型。
- 导出量化（PTQ）：训练后直接量化（如FP32→INT8），永久降低精度。部署友好，但需校准数据，适合低延迟推理（如边缘设备）。

量化方法	核心特点	适用场景
GPTQ	统一4-bit量化，最小化单层误差	极限压缩，适合显存受限环境
Unsloth动态量化	混合精度（部分层4-bit，关键层FP16）	最大程度保留模型性能
QLoRA（微调量化）	训练时4-bit量化，推理恢复FP16	降低训练成本，但部署仍需FP16
导出量化（PTQ）	训练后永久降低精度（如INT8/4-bit）	部署优化，减少推理成本

DeepSeek + Cursor使用： 物理世界中的小球碰撞

DeepSeek + Cursor使用

在 File -> Preferences -> Cursor Settings 中设置

deepseek-r1 和 deepseek-v3模型

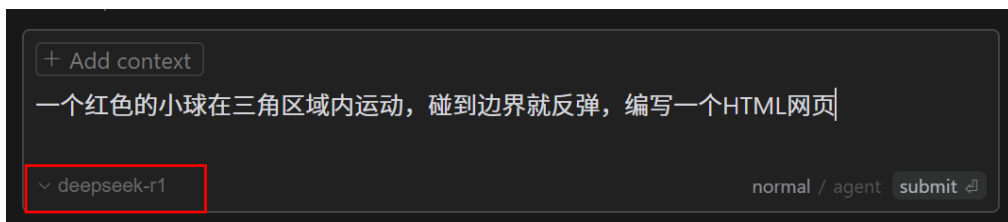
在OpenAI API Key中进行设置，这里是采用OpenAI的协议，可以使用自定义的模型

OpenAI Key = sk-

Q2gN9CgZOz9jrjCCHkijalkUyaXpHS6xssmmkl327kkib0G

OpenAI Base URL = <http://chatapi.littlewheat.com/v1>

设置好 deepseek-r1 和 deepseek-v3模型之后，可以在Cursor中选择该模型进行编程



Model Names

Add new models to Cursor. Often used to configure the latest OpenAI models or OpenRouter models.

- ☐ claude-3.5-sonnet
- ☐ cursor-small
- ☒ deepseek-r1
- ☒ deepseek-v3

OpenAI API Key

With your Cursor Pro subscription, you do not need to use your own OpenAI key unless you're using custom models.

Dismiss [Turn Off OpenAI Key](#)

You can put in [your OpenAI key](#) to use Cursor at public API costs. Note: this can cost more than pro and won't work for custom model features.

..... [Verify →](#)

Override OpenAI Base URL ▾

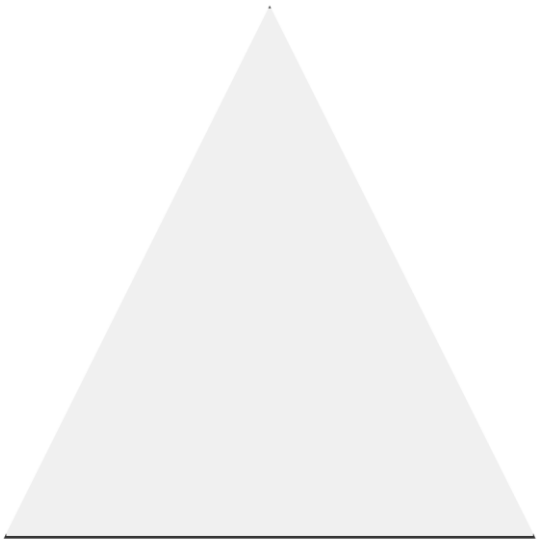
<http://chatapi.littlewheat.com/v1>

[Save](#)

[Reset to default](#)

CASE：物理世界中的小球碰撞

一个红色的小球在三角区域内运动，碰到边界就反弹，
编写一个HTML网页



deepseek-r1 会进行很长时间的思考，然后编写出HTML代码。
编写的HTML代码运行可能不符合要求，需要进一步调整

<think>

好的，我现在需要帮助用户创建一个在三角区域内反弹的小球的HTML页面。首先，我得考虑如何实现这个效果。用户想要一个红色小球在三角形内运动，碰到边界反弹。我需要用HTML5的Canvas来绘制图形和动画。

首先，我得确定三角形的边界。三角形通常有三个边，但检测碰撞可能会比较复杂。因为Canvas的路径是矢量，可能需要用数学方法来判断小球是否碰到三角形的边。或者，我可以将三角形视为三个线段，分别检测碰撞。

接下来是小球的运动。小球有位置 (x,y) 和速度 (dx,dy)，每次动画帧更新位置。当碰到边界时，速度方向改变。但是，三角形的边界是斜边，所以反弹方向不能简单取反，需要计算反射向量。

这可能会比较复杂。用户可能希望简化处理，比如将三角形视为三个墙壁，每次碰撞后根据碰到的边反转速度的相应分量。或者，可能需要计算小球到各边的距离，判断是否发生碰撞，然后调整速度方向。

考虑到用户可能希望代码简单，我先考虑一个等边三角形，顶点在上方，底边水平。这样三条边分别是左斜边、右斜边和底边。对于左右斜边，碰撞检测可能需要计算点是否在边的附近，并调整速度方向。

但这样做可能数学计算较多。或者，可以使用三个直线方程，检测小球的中心是否接近某条边，并计算反射方向。

另一个思路是使用边界框，将三角形的三个边作为反射面，当小球碰撞时，根据边的方向调整速度。例如，底边是水平的，碰撞时反转dy；左右边是斜的，碰撞时需要计算反射。

这可能涉及到向量运算，比如计算反射向量。假设边的法线向量，然后根据入射速度计算反射速度。

不过，这样计算起来可能有点复杂，特别是对于初学者。有没有更简单的方法？比如，将三角形视为由三个线段组成，每次检测小球与这三个线段的碰撞，然后反弹。

或者，使用边界的斜率和小球的运动方向来判断碰撞。例如，当小球碰到左斜边时，根据边的斜率调整速度分量。

不过，这样可能需要较多的几何计算。为了简化，我可以假设三角形是一个等边三角形，三个边的方程已知，然后检测小球的中心是否接近某条边，并在碰撞时调整速度方向。

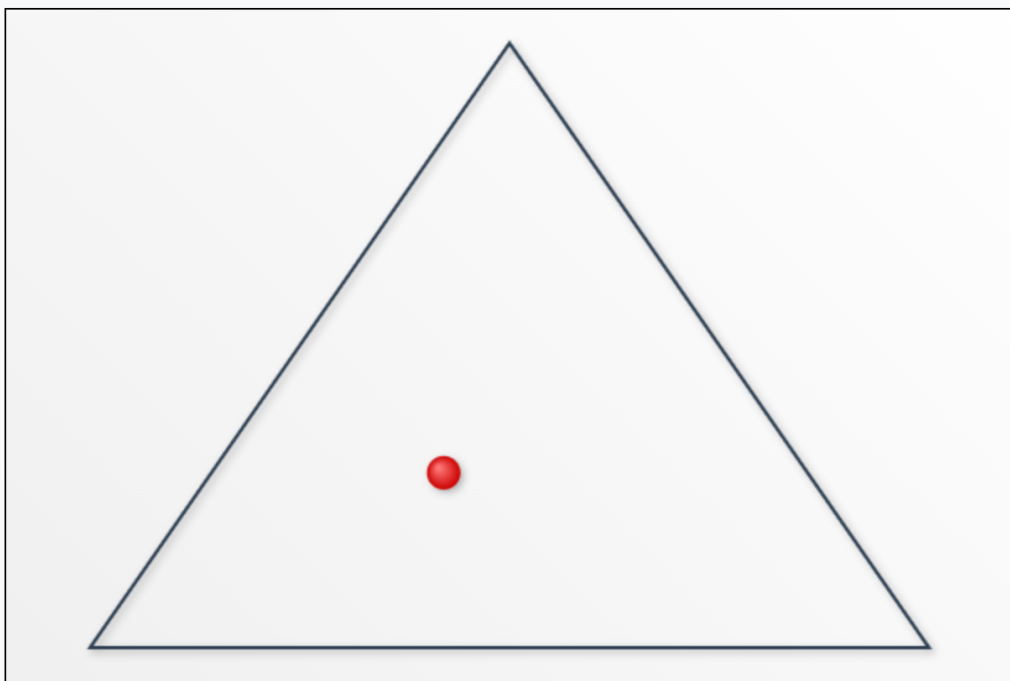
现在，我需要考虑如何表示三角形的边。例如，左斜边可能是从左上角到右下角，右斜边是从右上角到右下角，底边是水平的。假设三角形的顶点在画布的上中点，底边在底部。

接下来，小球的初始位置应该在三角形内部。当小球移动时，检查它是否超出了三角形的任何一边。如果超出，则反弹。

但如何判断点是否在三角形内？或者，当小球的中心碰到边时，视为碰撞。这可能需要进

CASE：物理世界中的小球碰撞

基于之前的思考，帮我完善HTML



红色小球在三角形区域内持续反弹运动

HTML样式有了调整，但是小球还是会飞出三角区域，需要进一步调整

<think>

好吧，用户希望完善之前的HTML，但当前的代码已经实现了基本功能。首先，我需要仔细检查代码，看看哪些地方可以改进。注意到用户提到之前的功能是正确的，所以不能破坏现有逻辑。

首先，界面方面，原生的canvas边框比较单调，可以考虑添加背景色或者渐变效果，让画面更吸引人。比如给画布加个渐变背景，这样视觉效果更好。

然后，动画流畅度方面，当前的移动速度是固定的，用户可能想要更自然的运动。可以考虑调整速度参数，比如使用小数，让运动更平滑。或者添加随机的速度变化，但要注意不能改变原有反弹逻辑。

界面文字提示也是一个改进点。可以在页面下方添加说明文字，比如使用CSS在body中添加一个div，说明这是三角区域反弹球，这样用户更清楚演示内容。

另外，小球的外观可以优化，比如添加阴影效果或者高光，让它看起来更立体。在drawBall函数里，用ctx.shadow来增加立体感。

代码可读性方面，可以添加更多注释，尤其是碰撞检测的部分，虽然用户可能已经了解，但清晰的注释有助于维护。比如在checkCollisionWithEdge函数中解释参数A和B代表边的两个端点。

检查是否存在中文乱码，确保HTML的meta标签有设置UTF-8。原来的代码没有<meta charset>，需要补充，这样中文注释和可能的文本就不会乱码。

最后，考虑响应式布局，让画布在不同设备上居中显示。可以通过CSS的flex布局让canvas居中，提升用户体验。

综合这些点，逐步修改代码，确保每一处改动都不影响原有功能。比如添加meta标签，修改背景样式，增加文字提示，美化小球外观，并调整速度参数。同时保持代码结构清晰，注释准确。

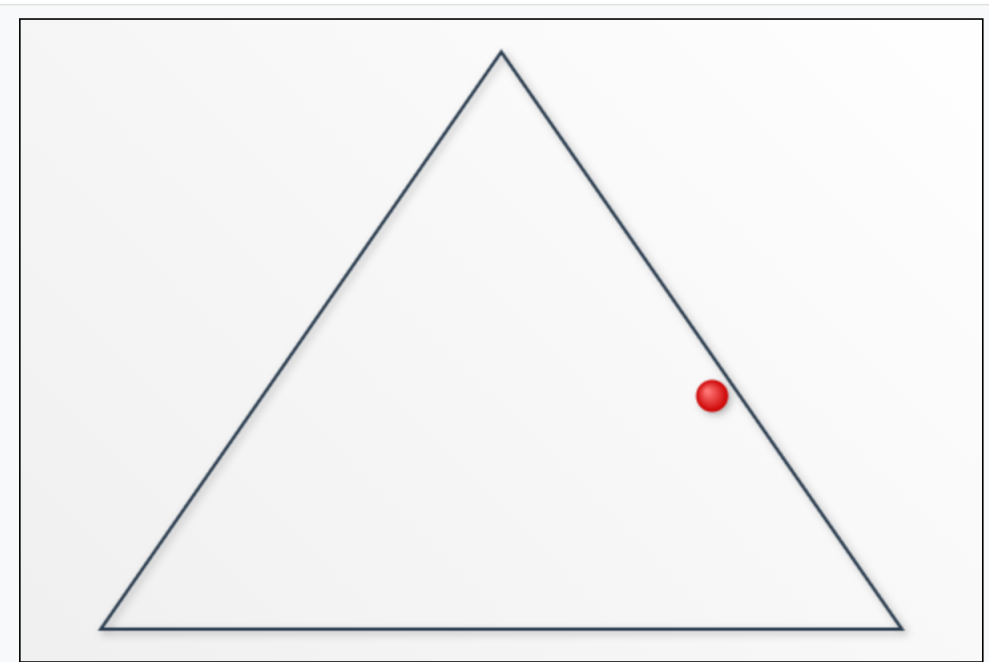
</think>

我将在原有功能不变的基础上做以下改进优化：

```
<?index.html +168 -5 ✓>
<!DOCTYPE html>
<html>
<head>
<style>
```

CASE：物理世界中的小球碰撞

小球弹了之后，弹出去了啊，帮我检查代码



红色小球在三角形区域内持续反弹运动

直接告诉deepseek-r1遇到的错误，修正后，小球会在三角区域内进行运动

<think>

好的，用户反馈说小球碰撞后弹出去了，我需要仔细检查代码中的碰撞检测逻辑。首先，查看checkCollisionWithEdge函数，注意到计算法线向量的方式可能有问题。

原算法线方向的部分，使用的是最近的点的坐标减去起点A的坐标，再除以边长度，但这种计算是否垂直于边呢？假设边的向量是 (B.x - A.x, B.y - A.y)，那么与其垂直的向量应该是 (B.y - A.y, -(B.x - A.x)) 或者相反方向。原代码中计算normal的x分量是(nearest.y - A.y)/edgeLength，这等于边的y分量除以长度，这可能错误地将边的方向当成了法线方向。

正确的法线应该使用边的垂直向量。比如，边向量是 (edgeX, edgeY)，则法线应该是 (edgeY, -edgeX) 方向，并进行单位化。原代码中使用的是从最近点减去A点后的处理，这可能导致了法线方向错误，导致反射后的速度方向不正确，出现弹出问题。

需要修改normal的计算方式，使用边向量取垂直向量，确保法线方向正确，从而正确反射速度，这样小球就不会弹出边界了。

</think>

经过检查，碰撞检测中的法线方向计算存在微小误差。我们需要修正法线向量计算方式：

<> index.html +139 -2 •

```
<!DOCTYPE html>
<html>
<head>
  <style>
    canvas {
      border: 1px solid #000;
      background: linear-gradient(45deg, #f0f0f0, #ffffff);
    }
  </style>
</head>
</html>
```

问题原因与修正说明：

1. 原法线方向计算错误：

- 旧代码使用的计算方式会导致法线方向与实际边向量方向不一致
- 修正后直接通过边向量计算法向量，确保方向准确

2. 反射公式正确性验证：

- $normal.x = edge.y / edgeLength$ 和 $normal.y = -edge.x / edgeLength$

CASE：物理世界中的小球碰撞

考虑小球的重力，以及三角区域的支撑力，小球的弹力 进行完善

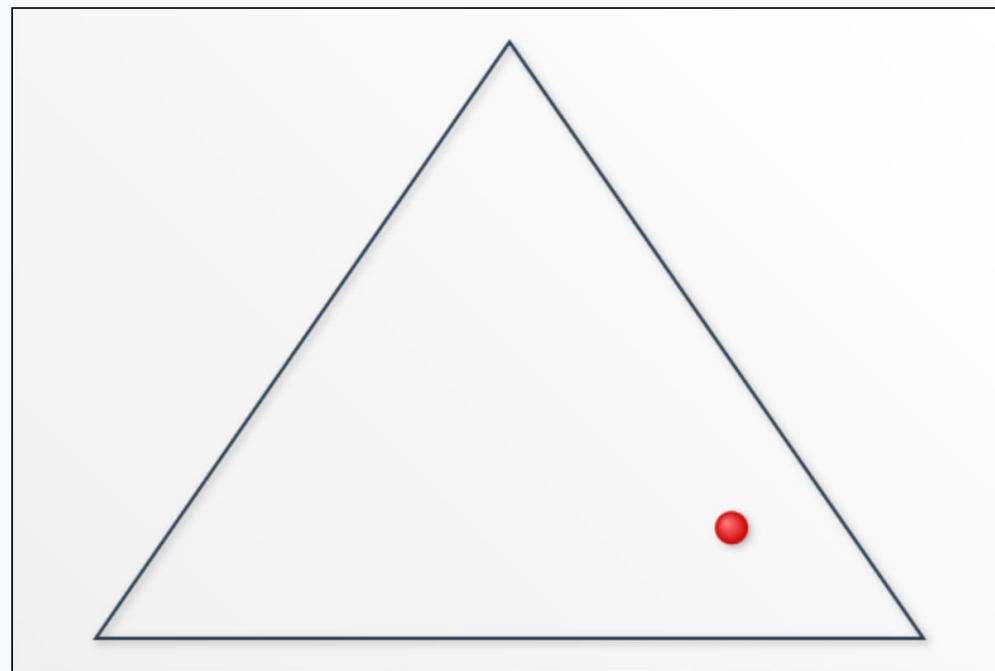
每次运行，小球可以从随机的任意方向抛出

小球的弹力大一些

最后到地面都垂直弹跳了，不是应该还会滚动么，就是有水平的速度

下面增加一个刷新按钮，可以重新执行这个HTML

中文显示是乱码，另外考虑到手机显示，页面需要做自适应



红色小球在三角形区域内持续反弹运动

重新开始

<http://case.wucaai.com/ball/>

Summary

Thinking: 当我们写程序的时候, 什么时候用 DeepSeek-V3-0324, 什么时候用 DeepSeek-R1?

V3-0324: 日常编程、快速开发、前端代码生成、常规脚本任务。

R1: 数学密集型计算、复杂算法、代码逻辑深度优化、需要推理过程的任务。

=> 更擅长复杂算法实现, 能优化逻辑并减少错误

DeepSeek私有化部署

私有化部署（DeepSeek模型）

DeepSeek-R1模型

Model	#Total Params	#Activated Params	Context Length	Download
DeepSeek-R1-Zero	671B	37B	128K	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1
DeepSeek-R1	671B	37B	128K	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Zero

DeepSeek-R1蒸馏模型

Model	Base Model	Download
DeepSeek-R1-Distill-Qwen-1.5B	Qwen2.5-Math-1.5B	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B
DeepSeek-R1-Distill-Qwen-7B	Qwen2.5-Math-7B	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-7B
DeepSeek-R1-Distill-Llama-8B	Llama-3.1-8B	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-14B
DeepSeek-R1-Distill-Qwen-14B	Qwen2.5-14B	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-14B
DeepSeek-R1-Distill-Qwen-32B	Qwen2.5-32B	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B
DeepSeek-R1-Distill-Llama-70B	Llama-3.3-70B-Instruct	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Llama-70B

私有化部署（DeepSeek模型）

目前开放出来的1.5B、7B、14B等模型是Qwen/llama借助R1推理强化调出来的“蒸馏”版本，不是真正的R1。真正的DeepSeek-R1是671B全量版

deepseek-r1:1.5b——1-2G显存

deepseek-r1:7b——6-8G显存

deepseek-r1:8b——8G显存

deepseek-r1:14b——10-12G显存

deepseek-r1:32b——24G-48显存

deepseek-r1:70b——96G-128显存

deepseek-r1:671b——496GB



<https://modelscope.cn/search?search=deepseek>

私有化部署（DeepSeek模型）

Category	Benchmark (Metric)	Claude-3.5-Sonnet-1022	GPT-4o 0513	DeepSeek V3	OpenAI o1-mini	OpenAI o1-1217	DeepSeek R1
	Architecture	-	-	MoE	-	-	MoE
	# Activated Params	-	-	37B	-	-	37B
	# Total Params	-	-	671B	-	-	671B
English	MMLU (Pass@1)	88.3	87.2	88.5	85.2	91.8	90.8
	MMLU-Redux (EM)	88.9	88	89.1	86.7	-	92.9
	MMLU-Pro (EM)	78	72.6	75.9	80.3	-	84
	DROP (3-shot F1)	88.3	83.7	91.6	83.9	90.2	92.2
	IF-Eval (Prompt Strict)	86.5	84.3	86.1	84.8	-	83.3
	GPQA-Diamond (Pass@1)	65	49.9	59.1	60	75.7	71.5
	SimpleQA (Correct)	28.4	38.2	24.9	7	47	30.1
	FRAMES (Acc.)	72.5	80.5	73.3	76.9	-	82.5
	AlpacaEval2.0 (LC-winrate)	52	51.1	70	57.8	-	87.6
	ArenaHard (GPT-4-1106)	85.2	80.4	85.5	92	-	92.3

DeepSeekR1 在英文上表现出色

私有化部署（DeepSeek模型）

Category	Benchmark (Metric)	Claude-3.5-Sonnet-1022	GPT-4o 0513	DeepSeek V3	OpenAI o1-mini	OpenAI o1-1217	DeepSeek R1
Code	LiveCodeBench (Pass@1-COT)	33.8	34.2	-	53.8	63.4	65.9
	Codeforces (Percentile)	20.3	23.6	58.7	93.4	96.6	96.3
	Codeforces (Rating)	717	759	1134	1820	2061	2029
	SWE Verified (Resolved)	50.8	38.8	42	41.6	48.9	49.2
	Aider-Polyglot (Acc.)	45.3	16	49.6	32.9	61.7	53.3
Math	AIME 2024 (Pass@1)	16	9.3	39.2	63.6	79.2	79.8
	MATH-500 (Pass@1)	78.3	74.6	90.2	90	96.4	97.3
	CNMO 2024 (Pass@1)	13.1	10.8	43.2	67.6	-	78.8
Chinese	CLUEWSC (EM)	85.4	87.9	90.9	89.9	-	92.8
	C-Eval (EM)	76.7	76	86.5	68.9	-	91.8
	C-SimpleQA (Correct)	55.4	58.7	68	40.3	-	63.7

DeepSeek-R1 在代码、数学、中文上表现出色

私有化部署（DeepSeek模型）

Model	AIME 2024 pass@1	AIME 2024 cons@64	MATH-500 pass@1	GPQA Diamond pass@1	LiveCodeBench pass@1	CodeForces rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16	26.7	78.3	65	38.9	717
o1-mini	63.6	80	90	60	53.8	1820
QwQ-32B-Preview	44	60	90.6	54.5	41.9	1316
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80	89.1	49	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70	86.7	94.5	65.2	57.5	1633

DeepSeek-R1蒸馏模型的表现优于GPT-4o及Claude-3.5-Sonnet闭源模型

私有化部署（代码模型）

Model	#Total Params	#Active Params	Context Length	Download
DeepSeek-Coder-V2-Lite-Base	16B	2.4B	128k	https://modelscope.cn/models/deepseek-ai/DeepSeek-Coder-V2-Lite-Base
DeepSeek-Coder-V2-Lite-Instruct	16B	2.4B	128k	https://modelscope.cn/models/deepseek-ai/DeepSeek-Coder-V2-Lite-Instruct
DeepSeek-Coder-V2-Base	236B	21B	128k	https://modelscope.cn/models/deepseek-ai/DeepSeek-Coder-V2-Base
DeepSeek-Coder-V2-Instruct	236B	21B	128k	https://modelscope.cn/models/deepseek-ai/DeepSeek-Coder-V2-Instruct

基于DeepSeekMoE框架发布了拥有160亿和2360亿参数的DeepSeek-Coder-V2。

其中，激活参数仅为24亿和210亿，这包括了基础模型和指令模型。

vllm使用

vllm使用：是由伯克利大学 LMSYS 组织开源的LLM高速推理框架，用于提升LLM的吞吐量与内存使用效率。它通过 PagedAttention 技术高效管理注意力键和值的内存，并结合连续批处理技术优化推理性能。vLLM 支持量化技术、分布式推理、与 Hugging Face 模型无缝集成等功能

```
vllm serve deepseek-ai/DeepSeek-R1-Distill-Qwen-32B --tensor-parallel-size 2 --max-model-len 32768 --enforce-eager
```

- vllm serve, 启动 vLLM 推理服务的命令
- deepseek-ai/DeepSeek-R1-Distill-Qwen-32B, Hugging Face 模型库中的模型名称, vLLM 会尝试从 HF 下载模型
- --tensor-parallel-size 2, 启用张量并行, 在 2 个 GPU 上分布式运行模型 (适合 32B 大模型)
- --max-model-len 32768, 设置模型的最大上下文长度 (32K tokens), 确保能处理长文本。
- --enforce-eager, 禁用 CUDA Graph 优化 (可能在某些环境下更稳定, 但性能稍低)

Vllm使用

Thinking: 如果我在本地的ubuntu下面有 /root/autodl-tmp/models/tcl90/deepseek-r1-distill-qwen-32b-gptq-int4, 如何使用vllm进行推理?

```
vllm serve /root/autodl-tmp/models/tcl90/deepseek-r1-distill-qwen-32b-gptq-int4 --tensor-parallel-size 1 --max-model-len 32768 --enforce-eager --quantization gptq --dtype half
```

关键改动: 指定本地路径: 替换 HF 模型名为你的本地路径。

--quantization gptq: 显式声明使用 GPTQ 量化。

--dtype: 设为 half (FP16) 或 auto (自动选择), 因为 GPTQ 本身是 4-bit, 但计算时需指定中间精度。

```
Loading safetensors checkpoint shards: 0% Completed | 0/4 [00:00<?, ?it/s]
Loading safetensors checkpoint shards: 25% Completed | 1/4 [00:00<00:02, 1.13it/s]
Loading safetensors checkpoint shards: 50% Completed | 2/4 [00:02<00:02, 1.22s/it]
Loading safetensors checkpoint shards: 75% Completed | 3/4 [00:03<00:01, 1.22s/it]
Loading safetensors checkpoint shards: 100% Completed | 4/4 [00:04<00:00, 1.17s/it]
Loading safetensors checkpoint shards: 100% Completed | 4/4 [00:04<00:00, 1.17s/it]
```

```
ERROR 03-25 11:02:05 [core.py:340] RuntimeError: CUDA out of memory occurred when warming up sampler
ase try lowering `max_num_seqs` or `gpu_memory_utilization` when initializing the engine.
ERROR 03-25 11:02:05 [core.py:340]
CRITICAL 03-25 11:02:05 [core_client.py:269] Got fatal signal from worker processes, shutting down. S
cause issue.
```

Vllm使用

```
vllm serve /root/autodl-tmp/models/tclf90/deepseek-r1-distill-qwen-32b-gptq-int4 --tensor-parallel-size 1 --max-model-len 4096 --quantization gptq --dtype half --gpu-memory-utilization 0.8 --max-num-seqs 8 --enforce-eager
```

```
INFO 03-25 11:06:45 [loader.py:429] Loading weights took 4.84 seconds
INFO 03-25 11:06:45 [gpu_model_runner.py:1176] Model loading took 18.1678 GB and 5.752492 seconds
ERROR 03-25 11:06:47 [core.py:340] EngineCore hit an exception: Traceback (most recent call last):
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/engine/core.py", line 332, in run
_engine_core
ERROR 03-25 11:06:47 [core.py:340]     engine_core = EngineCoreProc(*args, **kwargs)
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/engine/core.py", line 287, in __i
nit__
ERROR 03-25 11:06:47 [core.py:340]     super().__init__(vllm_config, executor_class, log_stats)
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/engine/core.py", line 62, in __in
it__
ERROR 03-25 11:06:47 [core.py:340]     num_gpu_blocks, num_cpu_blocks = self._initialize_kv_caches(
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/engine/core.py", line 124, in _in
italize_kv_caches
ERROR 03-25 11:06:47 [core.py:340]     kv_cache_configs = get_kv_cache_configs(vllm_config, kv_cache_specs,
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/core/kv_cache_utils.py", line 576
, in get_kv_cache_configs
ERROR 03-25 11:06:47 [core.py:340]     check_enough_kv_cache_memory(vllm_config, kv_cache_spec,
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/core/kv_cache_utils.py", line 468
, in check_enough_kv_cache_memory
ERROR 03-25 11:06:47 [core.py:340]     raise ValueError("No available memory for the cache blocks. ")
ERROR 03-25 11:06:47 [core.py:340] ValueError: No available memory for the cache blocks. Try increasing `gpu_memory_utilization` w
hen initializing the engine.
ERROR 03-25 11:06:47 [core.py:340]
CRITICAL 03-25 11:06:47 [core_client.py:269] Got fatal signal from worker processes, shutting down. See stack trace above for root
cause issue.
Killed
```

尽管我们已经将参数调整到非常保守的配置
(max-model-len=4096、gpu-memory-
utilization=0.8)，但 32B GPTQ 量化模型仍然
无法在 24GB GPU 上运行。

CASE: DeepSeek-R1-7B使用 (GPU部署)

CASE: DeepSeek-R1-7B使用

导入必要的库

```
from modelscope import AutoModelForCausalLM, AutoTokenizer
```

设置模型路径

```
model_name = "/root/autodl-tmp/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-7B"
```

加载模型

torch_dtype="auto" 自动选择合适的数据类型

device_map="cuda" 指定使用GPU加速

```
model = AutoModelForCausalLM.from_pretrained(  
    model_name,  
    torch_dtype="auto",  
    device_map="cuda" # 也可以设置为 "auto" 自动选择设备  
)
```

加载对应的分词器

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

设置用户输入的提示词

```
prompt = "帮我写一个二分查找法"
```

构建对话消息列表

```
messages = [  
    {"role": "system", "content": "You are a helpful assistant."},  
    {"role": "user", "content": prompt}  
]
```

将消息转换为模型可接受的格式

```
text = tokenizer.apply_chat_template(  
    messages,  
    tokenize=False,  
    add_generation_prompt=True  
)
```

CASE: DeepSeek-R1-7B使用

将文本转换为模型输入格式并移动到正确的设备上

```
model_inputs = tokenizer([text],
return_tensors="pt").to(model.device)
```

生成回复

```
generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=2000
)
```

提取生成的文本部分（去除输入部分）

```
generated_ids = [
    output_ids[len(input_ids):] for input_ids, output_ids in
zip(model_inputs.input_ids, generated_ids)
]
```

将生成的token ID解码为文本

skip_special_tokens=True 跳过特殊token

```
response = tokenizer.batch_decode(generated_ids,
skip_special_tokens=True)[0]
```

打印生成的回复

```
print(response)
```

嗯，我现在要学习一下二分查找法，也就是二分查找。我对这个算法不是很熟悉，但我知道它是一种高效的查找方法，特别是在有序数组中。让我先理清思路，然后一步步来。

首先，二分查找的基本思想是将一个数组不断对半分，然后在分好的两部分中继续查找，直到找到目标元素。这种方法的时间复杂度是 $O(\log n)$ ，这比线性查找要快很多，特别是当数组很大时。

那具体步骤是什么呢？我记得应该是先确定数组的中点，然后比较中点处的元素和目标元素的大小。如果中点处的元素等于目标，就找到了；如果目标比中点小，那么在左边继续查找；如果目标比中点大，则在右边继续查找。这个过程不断重复，直到找到目标或者数组遍历完毕。

举个例子，假设有一个有序数组：[1,3,5,7,9,11,13,15]，我要找数字7。首先，数组的长度是8，中点应该是第4个元素，也就是7的位置。所以直接找到了。但如果我要找的是数字6，那么中点是7，比6大，所以应该在左边继续查找。然后新的子数组是[1,3,5]，中点是3的位置，元素是5。6比5大，所以继续在右边查找。新的子数组是[5,7]，中点是6的位置，元素是7。但是6比7小，所以继续在左边查找，但左边只有一个元素5，比较后发现6不在数组中。

那么，二分查找的关键点在于数组必须是有序的，并且每次查找都要计算中点，并根据比较结果调整左右边界。那具体怎么在代码中实现呢？

我记得通常会用一个循环，初始时左边界left=0，右边界right=n-1。然后计算中点mid=(left+right)/2。如果数组[mid]等于目标，返回mid；否则，如果目标小于数组[mid]，则right=mid-1，继续在左边查找；否则，left=mid+1，继续在右边查找。直到left>right时，说明没找到。

那我来试着写一个伪代码：

```
function binary_search(array, target):
    left = 0
    right = len(array) - 1
    while left <= right:
        mid = (left + right) // 2
        if array[mid] == target:
            return mid
        elif array[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1 # 表示没找到
```

Ollama使用

Ollama使用

Ollama 官方主要支持 macOS 和 Linux，但 Windows用户也可以安装

方法1: 使用 WSL

Step1, 打开 PowerShell (管理员权限)，运行：

```
wsf --install
```

重启电脑后，WSL 会自动完成安装 (默认安装 Ubuntu)

Step2, 安装 Ollama

在 WSL 终端 (Ubuntu) 中运行：

```
curl -fsSL https://ollama.com/install.sh | sh
```

Step3, 启动 Ollama 服务

```
ollama serve
```

(保持此终端运行，另开一个新终端进行后续操作)

Ollama使用

方法 2: 直接下载 Windows 版

<https://ollama.com/>

安装后, Ollama 会作为服务运行 (可在任务管理器查看)

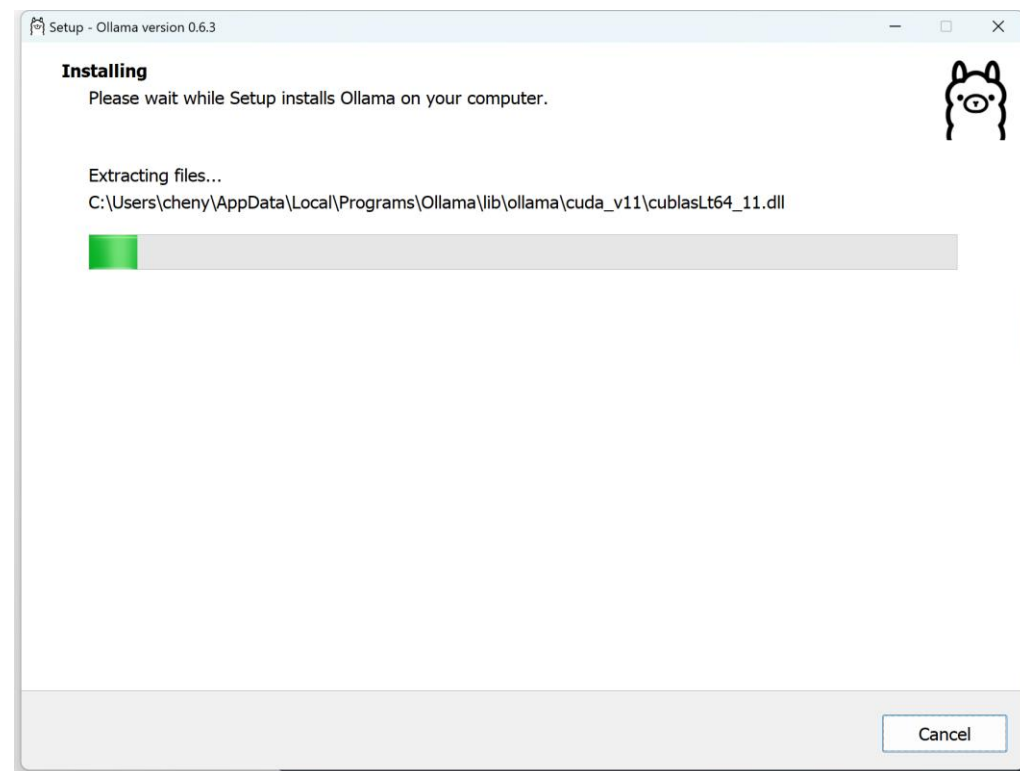


Get up and running with large
language models.

Run Llama 3.3, DeepSeek-R1, Phi-4, Mistral,
Gemma 3, and other models, locally.

Download ↓

Available for macOS,
Linux, and Windows



Ollama使用

Ollama官方模型库 <https://ollama.com/library>

下载 deepseek-r1:1.5b 模型

```
ollama pull deepseek-r1:1.5b
```

如果要删除该模型，可以使用

```
ollama rm deepseek-r1:1.5b
```

运行该模型，使用

```
ollama run deepseek-r1:1.5b
```

```
PS C:\Windows\System32> ollama run deepseek-r1:1.5b
>>> 你好啊
<think>

</think>
你好！很高兴见到你，有什么我可以帮忙的吗？
```

qwq

QwQ is the reasoning model of the Qwen series.

tools

32b

↓ 1.1M Pulls 8 Tags Updated 2 weeks ago

deepseek-r1

DeepSeek's first-generation of reasoning models with comparable performance to OpenAI-o1, including six dense models distilled from DeepSeek-R1 based on Llama and Qwen.

1.5b

7b

8b

14b

32b

70b

671b

↓ 31.5M Pulls 29 Tags Updated 7 weeks ago

CASE: Ollama使用 (使用Ollama REST API)

```
import requests

def query_ollama(prompt, model="deepseek-r1:1.5b"):
    url = "http://localhost:11434/api/generate"
    data = {
        "model": model,
        "prompt": prompt,
        "stream": False # 设置为 True 可以获取流式响应
    }

    response = requests.post(url, json=data)
    if response.status_code == 200:
        return response.json()["response"]
    else:
        raise Exception(f"API 请求失败: {response.text}")
```

使用示例

```
response = query_ollama("你好，请介绍一下你自己")
print(response)
```

<think>

您好！我是由中国的深度求索（DeepSeek）公司开发的智能助手 DeepSeek-R1。如您有任何任何问题，我会尽我所能为您提供帮助。

</think>

您好！我是由中国的深度求索（DeepSeek）公司开发的智能助手 DeepSeek-R1。如您有任何任何问题，我会尽我所能为您提供帮助。

Ollama 本身提供 REST API 接口，默认运行在 localhost:11434 运行 ollama serve 时，API 服务会自动启动。

CASE: Ollama使用 (FastAPI封装)

如果想添加额外的功能或自定义 API 端点, 可以使用 FastAPI 创建一个包装层:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import requests

app = FastAPI()

# 定义请求模型
class ChatRequest(BaseModel):
    prompt: str
    model: str = "deepseek-r1:1.5b"

# 允许跨域请求 (根据需要配置)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
@app.post("/api/chat")
async def chat(request: ChatRequest):
    ollama_url = "http://localhost:11434/api/generate"
    data = {
        "model": request.model,
        "prompt": request.prompt,
        "stream": False
    }

    response = requests.post(ollama_url, json=data)
    if response.status_code == 200:
        return {"response": response.json()["response"]}
    else:
        return {"error": "Failed to get response from Ollama"}, 500

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

7-ollama-fastapi.py

CASE：Ollama使用（FastAPI封装）

创建好API服务后，需要启动

python 7-ollama-fastapi.py

```
INFO: Started server process [34680]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

创建Python 客户端调用启动好的FastAPI（8000端口）

```
import requests

response = requests.post(
    "http://localhost:8000/api/chat",
    json={"prompt": "你好，请介绍一下你自己"}
)

print(response.json())
```

```
{'response': '<think>\n嗯，用户刚问我“你好”，然后让我介绍一下自己。看起来他可能是在测试我的响应，或者想了解我的身份。\\n\\n我应该先回应问候，接着详细介绍自己的身份和背景。要保持回答专业且全面，包括教育背景、工作经历和个人爱好。\\n\\n考虑到我是通过自然语言处理来思考这个问题的，我可以先列出一些关键点：教育背景、工作经验、兴趣爱好以及未来的职业规划。\\n\\n这样用户不仅得到了一个完整的回答，还能了解我为什么会选择这些信息，并且可能展示出我对自己的理解和自信。\\n\\n最后，用礼貌的结束语让用户放心。\\n</think>\n\n您好！我是由中国的深度求索（DeepSeek）公司开发的智能助手DeepSeek-R1。我是一个来自中国的人工智能助手，专注于帮助您解答各种问题。我的专业背景是计算机科学和人工智能，具备丰富的知识体系和解决问题的能力。如果您有任何疑问或需要进一步的帮助，请随时告诉我。'}
```

7-ollama-fastapi-python客户端.py

CASE: DeepSeek-R1使用 (API调用)

CASE：DeepSeek-R1使用-阿里代理

1、基本设置：

```
import dashscope
from dashscope.api_entities.dashscope_response import Role
# 设置 API key
dashscope.api_key = "your-api-key"
```

2、模型调用封装：

```
def get_response(messages):
    response = dashscope.Generation.call(
        model='deepseek-r1', # 使用 deepseek-r1 模型
        messages=messages,
        result_format='message' # 将输出设置为message形式
    )
    return response
```

3、情感分析实现：

```
review = '这款音效特别好 给你意想不到的音质。'
messages = [
    {"role": "system", "content": "你是一名舆情分析师，帮我判断产品口碑的正负向，回复请用一个词语：正向 或者 负向"},
    {"role": "user", "content": review}
]
response = get_response(messages)
print(response.output.choices[0].message.content)
```

正向

Summary

- 大模型API是连接AI能力的桥梁，让开发者无需关注底层架构即可调用前沿AI能力，极大拓展了技术应用的边界。
- Prompt工程是激活大模型潜力的钥匙

结构化设计（角色定义/分步指令/示例规范）

业务场景对齐（需求分析→Prompt迭代→效果验证）

性能优化技巧（温度系数/输出限制/上下文管理）

- LLM正在重塑开发范式，通过大模型API接口覆盖NLP/CV/多模态任务

打卡：大模型私有化部署与使用



对开源大模型进行私有化部署，选择适合的大模型，比如：

- deepseek-r1:1.5b, 7b, 8b, 14b, 32b, 70b等
- 或者qwen3系列：0.6b, 1.7b, 4b, 8b, 14b, 30b, 32b等

选择适合的部署方法

- Vllm高效部署
- Ollama部署
- Python本地部署（GPU）

部署任意的模型，选择其中一种部署的方法，调通即可，比如使用 ollama部署deepseek-r1:1.5b

提示词工程

Prompt原理

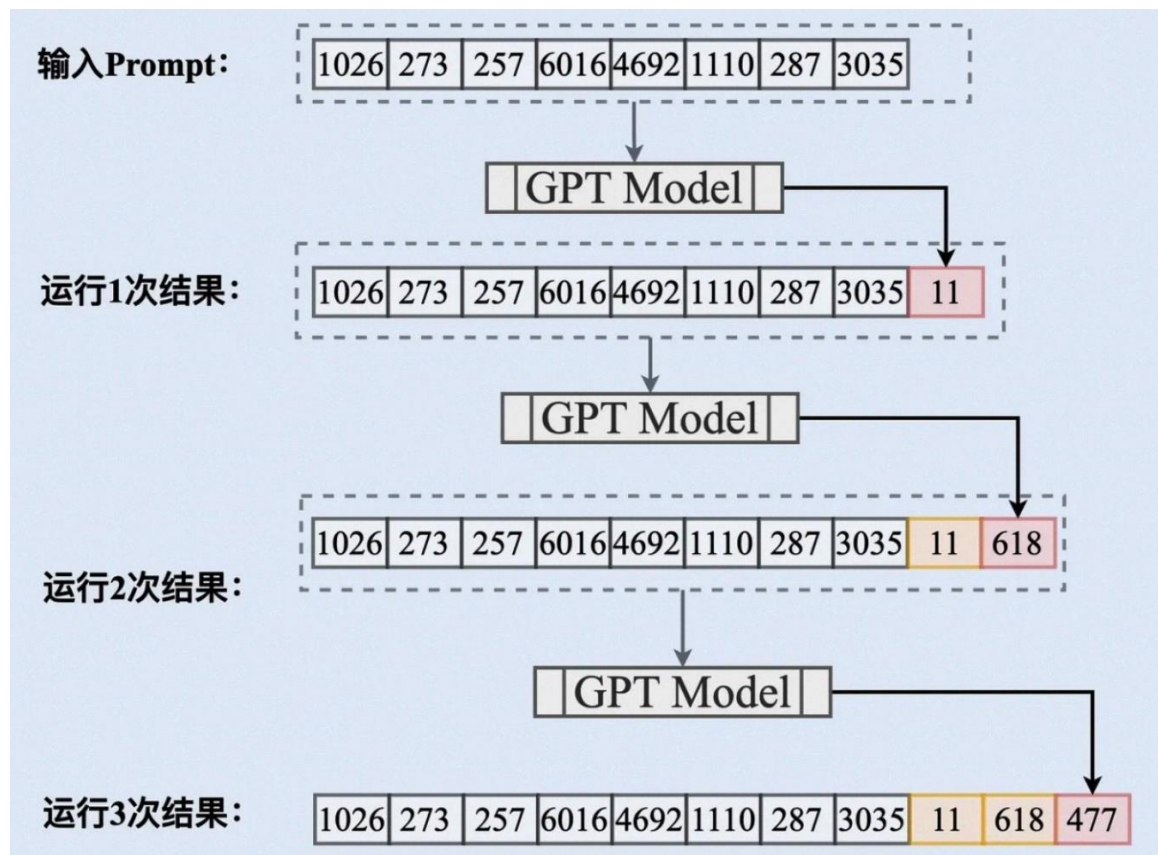
Prompt原理：

- GPT在处理Prompt时，GPT模型将输入的文本（也就是Prompt）转换为一系列的词向量。
- 然后，模型通过自回归生成过程逐个生成回答中的词汇。在生成每个词时，模型会基于输入的Prompt以及前面生成的所有词来进行预测。
- 这个过程不断重复，直到模型生成完整的回答

Thinking：你认为Prompt Engineering的重要性如何？

一个有效的Prompt可以：

- 提升AI模型给出的答案的质量
- 缩短与AI模型的交互时间，提高效率
- 减少误解，提高沟通的顺畅度



提示词策略差异

通用模型

- 需要显示引导推理步骤，比如通过CoT提示，否则可能会忽略关键逻辑
- 依赖提示词补偿能力短板，比如要求分步骤思考，提供few-shot参考示例等

推理模型

- 提示语更简洁，指需要明确任务目标 and 需求，因为模型已经内化了推理逻辑
- 无需逐步指导，模型会自动生成结构化推理过程。如果强行分步骤拆解，反而会降低其推理能力

提示词关键原则

模型选择

根据任务类型选择，而非模型热度

比如创意类任务选择通用模型，数学、物理、编程等理科推理类任务选择推理模型

提示词设计

通用模型：结构化、补偿性引导 => 缺什么，补什么

推理模型：简洁指令、聚焦目标、信任其内化能力 => 要什么，直接说

避免误区

不要对通用模型过度信任，如直接问复杂推理问题，需要分步骤进行

不要对推理模型进行启发式（简单）提问，推理模型会给你过于复杂的深度结果。

对于复杂问题，推理模型依然会有错误的可能，或存在忽略某些细节的可能，这时需要多次交互

Prompt 编写原则

Prompt 编写原则：

- **明确目标：**清晰定义任务，以便模型理解。
- **具体指导：**给予模型明确的指导和约束。
- **简洁明了：**使用简练、清晰的语言表达Prompt。
- **适当引导：**通过示例或问题边界引导模型。
- **迭代优化：**根据输出结果，持续调整和优化

一些有效做法：

- 强调，可以适当的重复命令和操作
- 给模型一个出路，如果模型可能无法完成，告诉它说"不知道"
- 尽量具体，对于专业性要求强的，少留解读空间（在你的专业领域中，把它看成孩子）



Prompt 编写原则

- **具体指导：** 给予模型明确的指导和约束。

示例 1：文本摘要生成任务目标

生成新闻文章的摘要。

不明确的指导：

请为这篇新闻文章生成摘要。

具体的指导：

请将以下新闻文章总结为3-4句话，包含主要事件、人物和时间地点。

示例 2：客户服务对话任务目标

回答客户关于订单状态的问题。

不明确的指导：

请回答客户关于订单状态的问题。

具体的指导：

请使用礼貌的语言回答客户关于订单状态的问题，提供具体的订单信息和预计到达时间。如果订单有任何问题，请提供解决方案或进一步的联系信息。

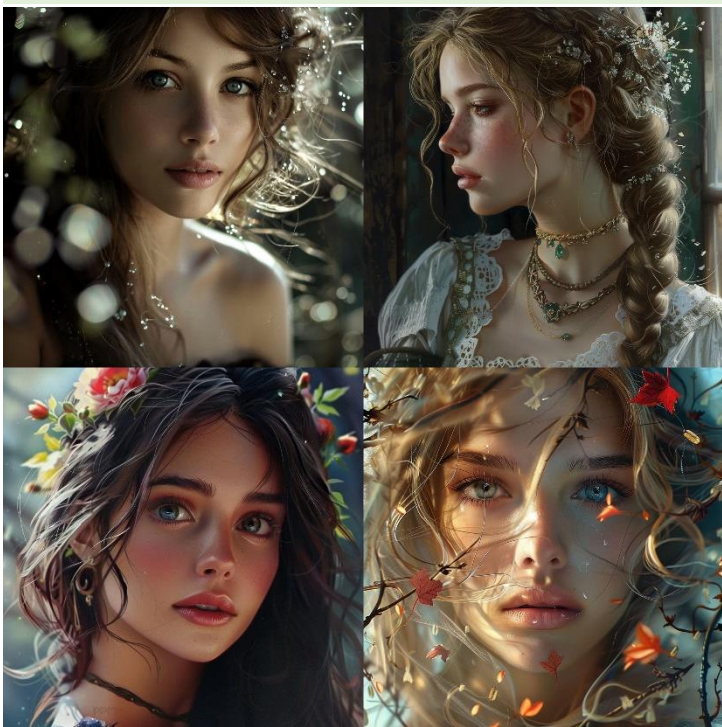
Prompt 编写原则

- **具体指导：**给予模型明确的指导和约束。

示例 3：生成 a beautiful girl 的图片

不明确的指导：

a beautiful girl



具体的指导：

anime girl with long dark hair , simple and elegant style, light silver and light gray, wavy resin sheets , in the style of digital painting, gongbi, realistic yet romantic, shiny/glossy, cute and dreamy, smooth lines, pure white background,



动漫少女，长黑发

数码画风，纯白底色

Prompt 编写原则

- **简洁明了：**使用简练、清晰的语言表达Prompt。

示例 1：文章续写任务目标

根据给定开头续写一段故事。

冗长的指导：

请基于下面提供的故事开头续写一段文字。续写时请保持与原文风格一致，注意故事的连贯性和合理性。希望续写的部分能够引人入胜，并且能够自然地衔接上文。如果有任何疑问，请尽量参照原文的风格进行。

简洁明了的指导：

根据以下开头续写故事：

示例 2：翻译任务任务目标

将一段英文文本翻译成中文。

冗长的指导：

请将以下英文文本翻译成中文。我希望翻译的内容准确，并且保持原文的意思。翻译时请注意语法和用词，尽量使用简洁明了的语言表达。如果有任何不确定的地方，请尽量保持原意，不要进行过多的自由发挥。下面是需要翻译的文本。

简洁明了的指导：

请将以下英文文本翻译成中文：

Thinking：具体指导与简洁明了是相对的，你是如何理解这两个原则的？

Prompt 编写原则

- **适当引导：**通过示例或问题边界引导模型。

示例 1：邮件撰写

TO DO：写一封请假邮件。

无引导的指导：

请写一封请假邮件。

如果有优秀范例模版，对于模型来说是很好的指导

有引导的指导：

请写一封请假邮件，内容包括请假原因、请假时间以及对工作的安排。

示例：

主题：请假申请

尊敬的[经理姓名]，

我因个人原因需要请假两天，从6月20日至6月21日。请假期间，我会确保手头工作交接给同事[同事姓名]，并保持电话畅通。如有紧急情况，可随时联系我。感谢您的理解和批准。

此致

敬礼

[您的姓名]

Prompt 编写原则

- **迭代优化：** 根据输出结果，持续调整和优化

帮我写一个 a+b 的 Python 函数

下面是一个简单的 Python 函数，用于计算两个数 a 和 b 的和：

```
def add(a, b):  
    return a + b
```

示例使用

```
result = add(3, 5)
```

```
print(result) # 输出: 8
```

需要考虑 a 和 b 不是数值的情况

为了处理 a 和 b 可能不是数值的情况，可以在函数中添加类型检查，并在参数不是数值时抛出一个适当的异常。以下是一个改进的版本：

```
def add(a, b):  
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):  
        raise TypeError("Both a and b must be numbers (int or float)")  
    return a + b
```

Prompt 编写框架（重要性排序）

Prompt的组成：

- 1.任务（Task）：始终以动词开始任务句子（如"生成", "给予", "写作"等），明确表达你的最终目标，可以有多个。
- 2.上下文（Context）：提供上下文背景、所处环境等。
- 3.示例（Exemplars）：提供具体的例子，用于可以提高输出质量。有时候我们没有示例，这时可以让GPT帮忙生成示例。
- 4.角色（Persona）：你希望AI扮演的角色。可以是具体的人，也可以是虚构的角色。
- 5.格式（Format）：你希望输出的格式，如表格、列表、段落等。
- 6.语气（Tone）：指定输出的语气，如正式、非正式、幽默等。

重要性排序：

任务 > 上下文 > 示例 > 角色 > 格式 > 语气

如果模型回答不满意，依次Check自己的Prompt内容进行调整

写prompt是一个不断尝试和调整的过程

Prompt 编写技巧

在了解Prompt编写原则的基础上，还有一些Prompt编写技巧：

- 1、限制模型输出的格式
- 2、使用分隔符区分输入的不同部分
- 3、提供样例
- 4、CoT 思维工具
- 5、面向不同的角色进行讲解

技巧1：限制模型输出的格式

比如让大模型输出JSON格式，可以结构化数据，方便后续解析

用户：请从财务报表中提取以下信息，包括：公司名称，股票代码，营收，净利润，毛利，总资产，总负债。并以JSON格式返回。

```
{  
  "公司名称": "伊利股份",  
  "股票代码": "600887",  
  "营收": "325亿元",  
  "净利润": "59.23亿元",  
  "毛利": "105.6亿元",  
  "总资产": "1516.2亿元",  
  "总负债": "910亿元"  
}
```

Prompt 编写技巧

技巧2：使用分隔符区分输入的不同部分

请把三个引号括起来的文本，分别总结成为一句话

"""文本内容1"""

"""文本内容2"""

在编写Prompt时，通过使用分隔符，可以清晰地区分不同部分的文本，提升文本的组织性和可读性。

技巧3：提供样例

针对有一定歧义或复杂的情况，给出具体示例方便大模型进行学习

请帮我对用户的评价进行分类，直接输出：正面/负面，并给出理由

示例1：

用户评价：这次开户真是太满意了，都一周了，客服还没有回复我

输出：负面，客户并不是真的满意，客服一周未回复，造成了不满。

示例2：

用户评价：广发银行的开户过程非常顺利，客服态度亲切。

输出：正面，开户过程顺利，客服态度好。

请回答如下问题：

用户评价：开户流程简洁明了，不需要繁琐的纸质材料，所有步骤都可以在线完成

Prompt 编写技巧

技巧4: CoT 思维工具

CoT (Chain of Thought, 思维链) 通过将复杂任务分解为多个简单的步骤, 帮助模型系统地思考并解决问题。

你是一个数学助手, 请根据以下步骤计算用户输入的金额。请将每个金额首先加上1000元, 接着减去500元, 然后乘以1.2输出计算结果, 以','作为分隔符进行返回。

你可以参考以下计算过程来帮助解决:

.....

对于输入: 2000, 3000, 4000

计算过程如下:

首先分别对输入的2000, 3000, 4000加上1000, 得到: 3000, 4000, 5000

然后将3000, 4000, 5000分别减去500, 得到: 2500, 3500, 4500

然后将2500, 3500, 4500分别乘以1.2, 得到: 3000, 4200, 5400

答案是: 3000, 4200, 5400

.....

输入: 1500, 2500, 3500

Prompt 编写技巧

你是一个客户服务助手，请按照以下步骤处理客户的投诉。首先，记录客户的投诉类型，然后确定处理优先级（高、中、低），接着分配给适当的部门，最后生成一个处理跟踪编号并直接输出。

计算过程示例：

""""

客户投诉：客服的态度冷淡，没有耐心解答我的问题。

处理过程如下：

首先记录客户的投诉类型，得到：客服态度差

然后确定处理优先级，得到：优先级高

然后分配给适当的部门，得到：客服部

最后生成一个跟踪编号，得到 KF10001

""""

客户投诉：我的账户意外冻结了。我并未收到任何提前通知或解释。

CoT（Chain of Thought，思维链）的作用：

- 系统化问题解决：将复杂问题分解为多个简单步骤，使解决过程更加有序和清晰。
- 提高透明度，减少错误：帮助理解和追踪每个决策点，减少整体错误发生的概率。
- 提升模型推理能力：帮助模型学习如何系统地分解和解决此类问题，提高在类似任务中的表现。

Prompt 编写技巧

技巧5：面向不同的角色进行讲解

把我当做五岁小朋友一样，向我解释超导体。

好的！让我们用一个简单的比喻来解释超导体。

想象一下，你有一条滑滑的滑滑梯。如果你把一个小球放在滑梯的顶端，小球会很快滑到滑梯的底端，对吗？这个滑滑梯很光滑，所以小球滑下来时不会有太多阻力。这就像一个超导体。

在电线上，电流就像小球在滑梯上滑动。通常，电线上有一些阻力，就像粗糙的滑滑梯，小球滑下时会慢一些。但在超导体里，就像一个超级光滑的滑

滑梯，小球（电流）可以一直滑下去而不会停，也没有任何阻力。这意味着电流可以非常快速、非常轻松地通过超导体。

所以，超导体就是一种可以让电流毫无阻力地通过的特殊材料，电流在其中就像小球在光滑的滑梯上一样，滑得非常快！

学会与AIGC沟通

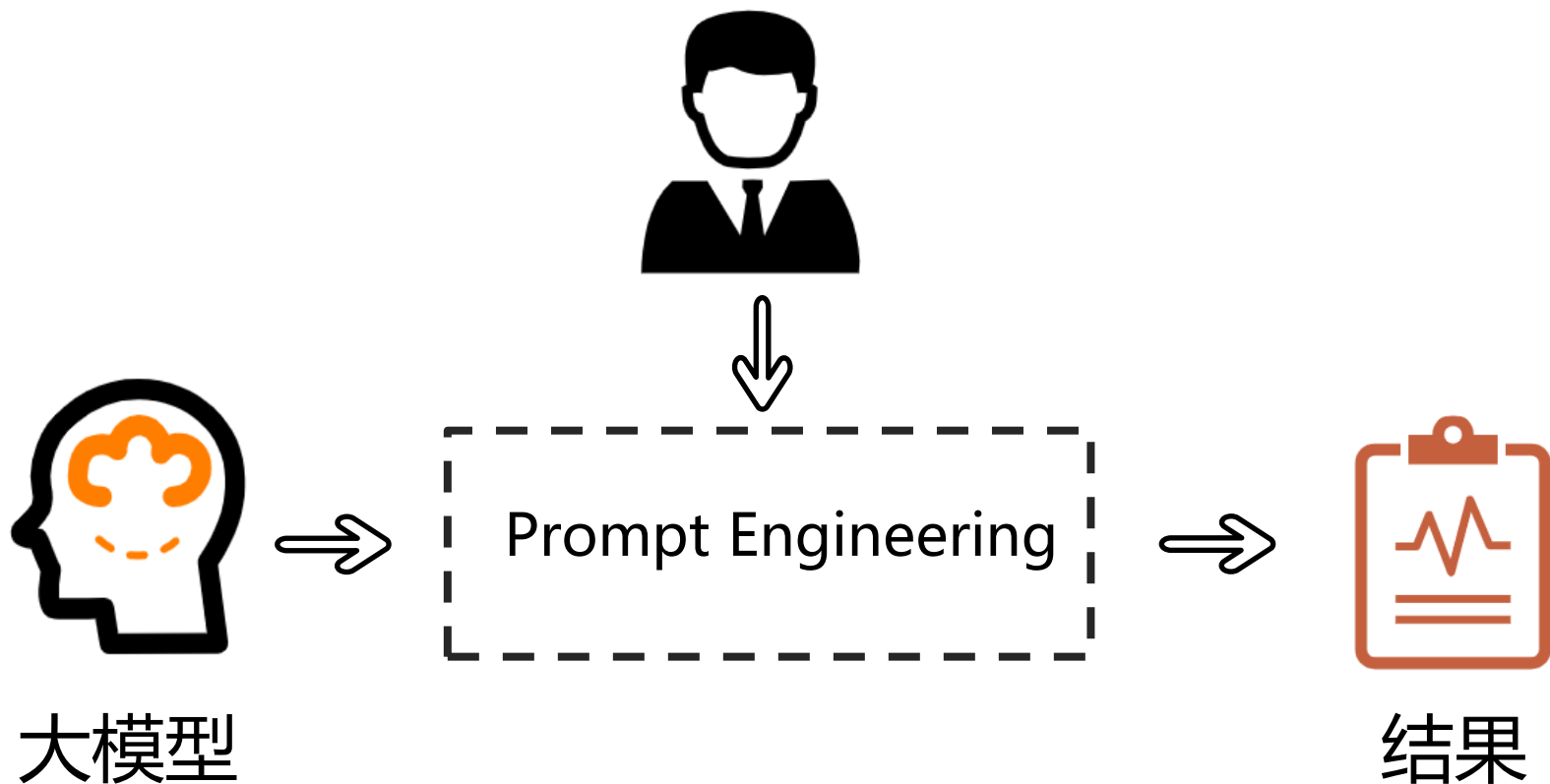
Prompt Engineering

未来每个人都是prompt engineer

任务结果 = 大模型 + Prompt

Thinking: 如何得到我们想要的结果

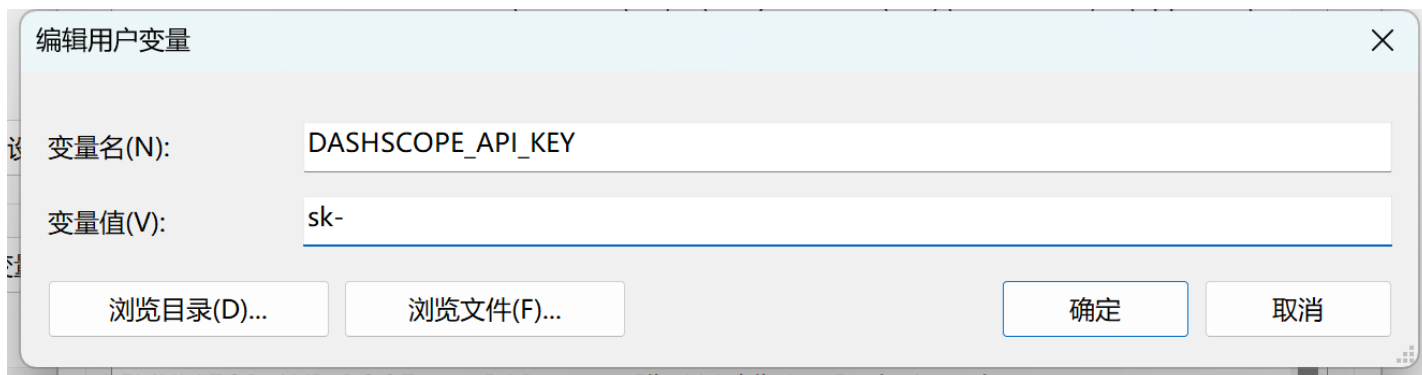
- 了解我们的需求
- 了解大模型的工作原理



Prompt实战

设置API Key

设置环境变量中的 DASHSCOPE_API_KEY



```
import dashscope
```

```
import os
```

```
# 从环境变量中获取 API Key
```

```
dashscope.api_key = os.getenv('DASHSCOPE_API_KEY')
```


CASE：使用提示词完成任务

Thinking：我们想让AI扮演电信的客服人员，如何识别用户的手机流量套餐的需求？

```
# 基于 prompt 生成文本
def get_completion(prompt, model="deepseek-v3"):
    messages = [{"role": "user", "content": prompt}] # 将 prompt 作为用户输入
    response = dashscope.Generation.call(
        model=model,
        messages=messages,
        result_format='message', # 将输出设置为message形式
        temperature=0, # 模型输出的随机性，0 表示随机性最小
    )
    return response.output.choices[0].message.content # 返回模型生成的文本
```

CASE：使用提示词完成任务

任务描述

```
instruction = """
```

你的任务是识别用户对手机流量套餐产品的选择条件。

每种流量套餐产品包含三个属性：名称，月费价格，月流量。

根据用户输入，识别用户在上述三种属性上的需求是什么。

```
"""
```

用户输入

```
input_text = """
```

办个100G的套餐。

```
"""
```

prompt 模版。instruction 和 input_text 会被替换为上面的内容

```
prompt = f"""
```

目标

```
{instruction}
```

用户输入

```
{input_text}
```

```
"""
```

调用大模型

```
response = get_completion(prompt)
```

```
print(response)
```

根据用户输入“办个100G的套餐”，可以识别出用户对手机流量套餐产品的选择条件如下：

1. **月流量**：用户明确需求是**100G**的月流量。
2. **名称**：用户没有提及具体的套餐名称，因此名称需求不明确。
3. **月费价格**：用户没有提及具体的月费价格，因此价格需求不明确。

总结：用户的主要需求是**月流量为100G**的套餐，对名称和月费价格没有明确要求。

CASE：JSON格式返回

Thinking：如何让AI返回JSON格式，更方便后续识别和使用

```
# 输出格式
```

```
output_format = ""
```

```
以 JSON 格式输出
```

```
""
```

```
# 稍微调整下咒语，加入输出格式
```

```
prompt = f""
```

```
# 目标
```

```
{instruction}
```

```
# 输出格式
```

```
{output_format}
```

```
# 用户输入
```

```
{input_text}
```

```
""
```

```
# 调用大模型，指定用 JSON mode 输出
```

```
response = get_completion(prompt)
```

```
print(response)
```

```
```json
```

```
{
```

```
 "月流量": "100G"
```

```
}
```

```
```
```

CASE：使用CoT分步骤推理

Thinking：如何审核客服的回答是否符合规范要求？

```
instruction = """
```

给定一段用户与手机流量套餐客服的对话，。

你的任务是判断客服的回答是否符合下面的规范：

- 必须有礼貌
- 必须用官方口吻，不能使用网络用语
- 介绍套餐时，必须准确提及产品名称、月费价格和月流量总量。上述信息缺失一项或多项，或信息与事实不符，都算信息不准确
- 不可以是话题终结者

已知产品包括：

经济套餐：月费50元，月流量10G

畅游套餐：月费180元，月流量100G

无限套餐：月费300元，月流量1000G

校园套餐：月费150元，月流量200G，限在校学生办理

```
"""
```

```
# 输出描述
```

```
output_format = """
```

如果符合规范，输出：Y

如果不符合规范，输出：N

```
"""
```

```
context = """
```

用户：你们有什么流量大的套餐

客服：亲，我们现在正在推广无限套餐，每月300元就可以享受1000G流量，您感兴趣吗？

```
"""
```

```
cot = ""
```

```
#cot = "请一步一步分析对话"
```

CASE：使用CoT分步骤推理

```
prompt = f"""
# 目标
{instruction}
{cot}

# 输出格式
{output_format}

# 对话上下文
{context}
"""

response = get_completion(prompt)
print(response)
```

CASE：使用Prompt调优Prompt

Thinking：如何使用Prompt优化Prompt？

```
user_prompt = """
```

做一个手机流量套餐的客服代表，叫小瓜。可以帮助用户选择最合适的流量套餐产品。可以选择的套餐包括：

经济套餐，月费50元，10G流量；

畅游套餐，月费180元，100G流量；

无限套餐，月费300元，1000G流量；

校园套餐，月费150元，200G流量，仅限在校生。"""

```
instruction = """
```

你是一名专业的提示词创作者。你的目标是帮助我根据需求打造更好的提示词。

你将生成以下部分：

提示词：{根据我的需求提供更好的提示词}

优化建议：{用简练段落分析如何改进提示词，需给出严格

批判性建议}

问题示例：{提出最多3个问题，以用于和用户更好的交流}

```
"""
```

```
prompt = f"""
```

```
# 目标
```

```
{instruction}
```

```
# 用户提示词
```

```
{user_prompt}
```

```
"""
```

```
response = get_completion(prompt)
```

```
print(response)
```

打卡：Prompt实战



结合你的业务场景，编写一个使用prompt驱动大模型完成任务，比如：

1) 使用提示词完成任务

比如 让AI扮演电信的客服人员，如何识别用户的手机流量套餐的需求？

2) 返回JSON格式，提取精确的内容

3) 使用CoT分步骤推理

4) 使用Prompt调优Prompt

- 完成以上任务之一即可，最好结合自己的业务场景，请将代码和结果截屏，发到微信群中！



Thank You
Using data to solve problems